

ROS2 Bridge Implementation – Simulink to YOLO

Node Communication

Author: Navnoor Kaur Virdi (225219832)

Date: 19th May 2026

Role: Scrum Master & Systems & Simulation Team Member

Project: E-waste Battery Extraction

1. What is this task and why is it needed?

In our project, a YOLO (computer vision) node detects batteries in camera images. It publishes the detection results (e.g., bounding boxes or simple messages) on a ROS2 topic called `/detections`.

Our robot (UR3) needs to receive those detections, decide how to move, and then send joint commands to the robot simulation. It also needs to publish its own current joint positions (state) so the YOLO node or other nodes can track what the robot is doing.

Therefore, we need a bridge in Simulink that:

- Subscribes to `/detections` (receives messages from YOLO)
- Publishes to `/joint_commands` (sends motion commands to the robot)
- Publishes to `/joint_states` (broadcasts the robot's current state)

This bridge is essential for closing the loop: Perception (YOLO) → Control (Simulink) → Robot (UR3 simulation) → Feedback (joint states) → back to control.

2. Implementation summary

I built a Simulink model (`ur3_ros2_bridge_v2.slx`) that contains:

- ROS 2 Subscribe block: listens to `/detections`, message type `std_msgs/String` (temporary; later replaced with custom detection message).
- Two ROS 2 Publish blocks: one for `/joint_commands`, one for `/joint_states`, both using `sensor_msgs/JointState`.
- Dummy JointState sources: Two ROS 2 Blank Message blocks configured to output a default `sensor_msgs/JointState`. This allows the Publish blocks to send valid messages even before real joint data is connected.
- Bus Selector + Display: Extracts the `data` field from the incoming `/detections` message and displays it to confirm reception.

All ROS2 blocks use the same domain ID (50) to ensure communication with the YOLO node.

A fake YOLO node (MATLAB script) publishes a string `"battery_detected"` on `/detections` every 2 seconds for testing.

3. How to build this model step by step

This section explains how to create the Simulink model from scratch. You only need to do this once – after that, you can reuse and modify the model as needed.

Prerequisites

- MATLAB R2025b or later with ROS Toolbox and Simulink installed.
- A working ROS2 environment (domain ID set to 50).

Step 1 – Create a new Simulink model

- In MATLAB, go to Home → New → Simulink Model → Blank Model.
- Save the model as `ur3_ros2_bridge_v2.slx`.

Step 2 – Add the Subscribe block (listens to /detections)

- Open Library Browser (Ctrl+Shift+L).
- Navigate to ROS Toolbox → ROS 2.
- Drag a Subscribe block into the model.
- Double-click it and set:
 - Topic source: Specify your own
 - Topic name: /detections
 - Message type: std_msgs/String
- Click OK.

Step 3 – Add a Bus Selector to extract the string data

- In the Library Browser, go to Simulink → Signal Routing.
- Drag a Bus Selector block into the model.
- Connect the Msg output of the Subscribe block to the input of the Bus Selector.
- Double-click the Bus Selector. In the left pane, select data and move it to the right pane. Click OK.

Step 4 – Add a Display block to see incoming messages

- In the Library Browser, go to Simulink → Sinks.
- Drag a Display block into the model.
- Connect the output of the Bus Selector to the Display block.

Step 5 – Add two Publish blocks (for /joint_commands and /joint_states)

- From ROS Toolbox → ROS 2, drag two Publish blocks into the model.
- Configure the first Publish block:
 - Topic source: Specify your own
 - Topic name: /joint_commands
 - Message type: sensor_msgs/JointState
- Configure the second Publish block the same way, but with Topic name: /joint_states.

Step 6 – Add dummy JointState sources for the Publish blocks

- From ROS Toolbox → ROS 2, drag two Blank Message blocks into the model.
- Double-click each Blank Message → Message type → select `sensor_msgs/JointState` → OK.
- Connect the Msg output of the first Blank Message to the input of the `/joint_commands` Publish block.
- Connect the Msg output of the second Blank Message to the input of the `/joint_states` Publish block.

Alternatively, you can use one Blank Message and branch its Msg output to both Publish blocks.

Step 7 – Set the ROS2 domain ID

- In the MATLAB command window, run:
`setenv('ROS_DOMAIN_ID', '50')`

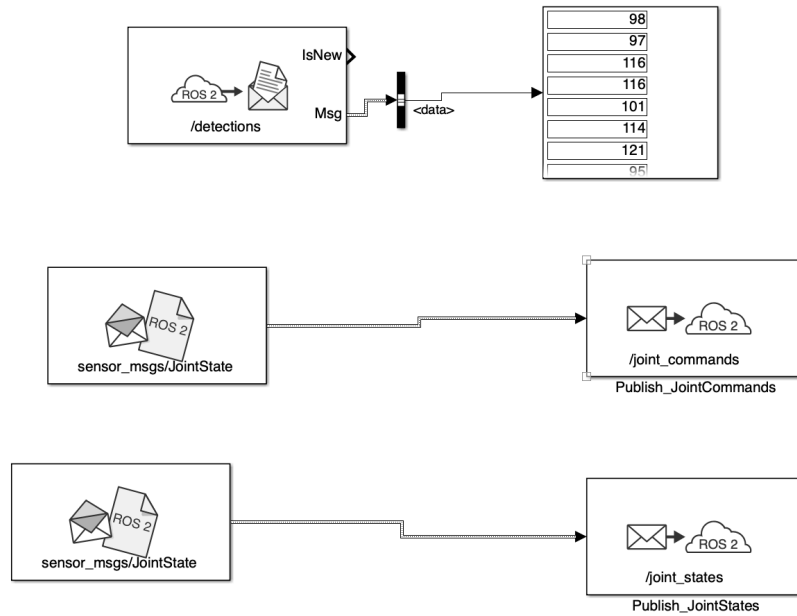
Step 8 – Save and run

- Save the model (Ctrl+S).
- Run the fake YOLO publisher script (provided in the previous section).
- Click Run in Simulink.
- Verify that the Display block shows ASCII numbers.

Result: You now have a working ROS2 bridge that can receive detections and publish joint commands and states. Later, you can replace the dummy Blank Message blocks with real joint data from your UR3 robot model.

3. Screenshots

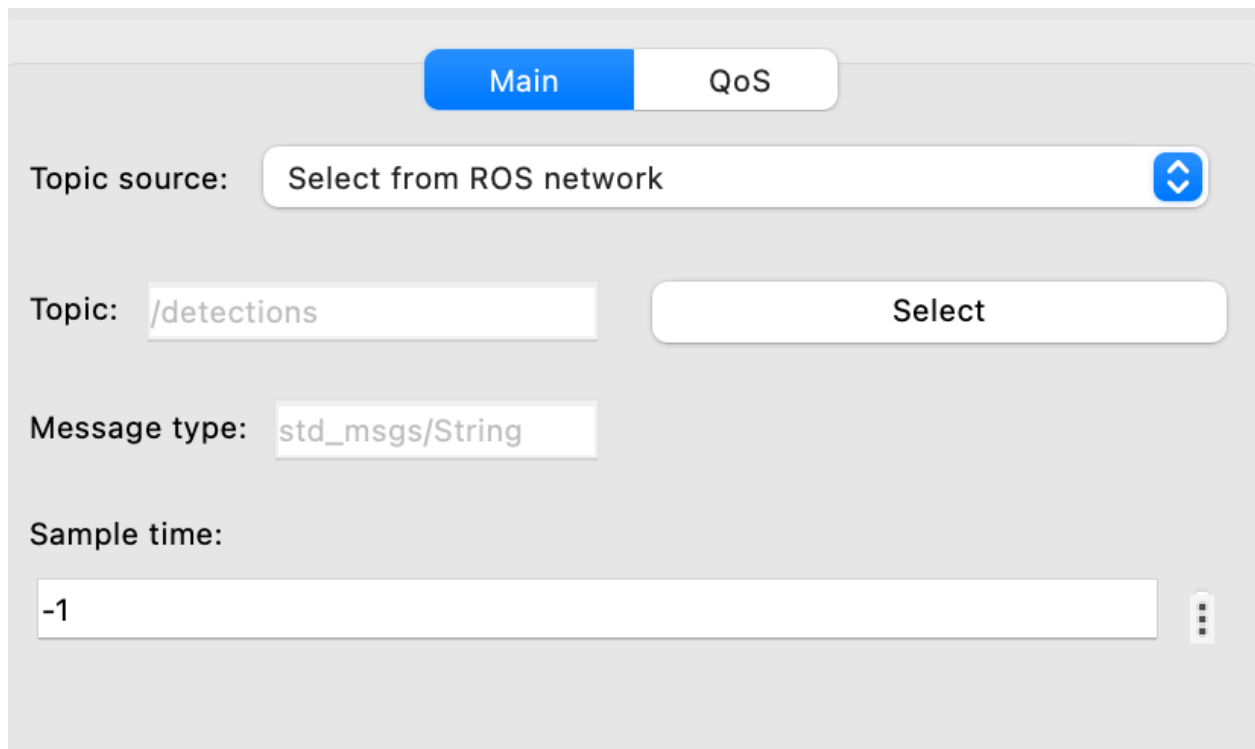
Screenshot 1: Simulink model overview



Simulink model `ur3_ros2_bridge_v2` with

1. Top Section: ROS2 Subscribe, Bus Selector and Display Message.
2. Bottom: Two ROS2 Publish blocks (bottom) and two Blank Messages.

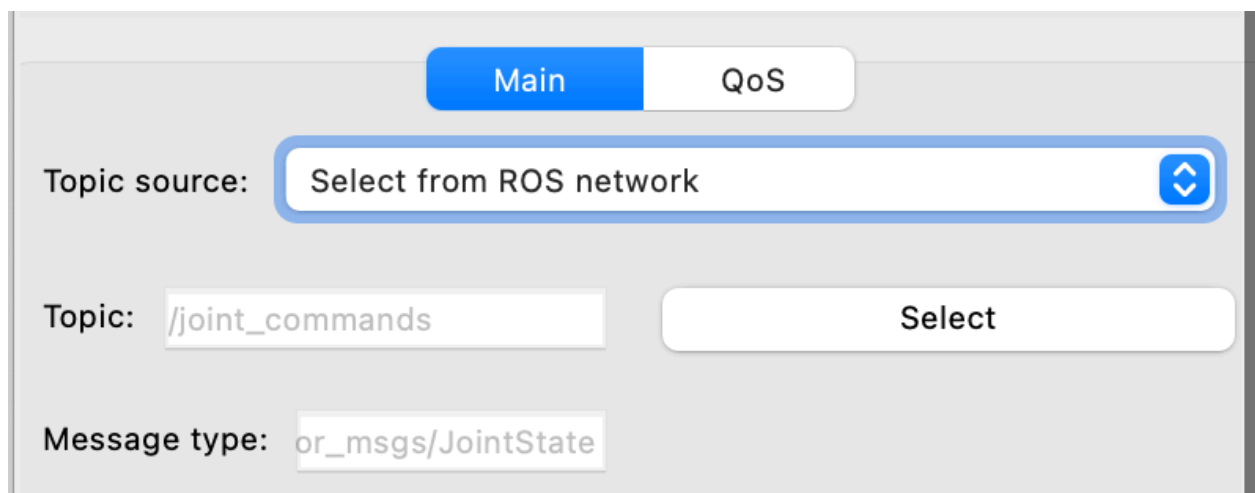
Screenshot 2: Subscribe block configuration



The screenshot shows the 'Main' tab of a configuration window. At the top, there are two tabs: 'Main' (active) and 'QoS'. Below the tabs, the 'Topic source' is set to 'Select from ROS network' with a dropdown arrow. The 'Topic' field contains '/detections' and a 'Select' button is to its right. The 'Message type' field contains 'std_msgs/String'. The 'Sample time' field contains '-1' and a vertical ellipsis button to its right.

Subscribe block configuration

Screenshot 3: Publish block configurations



The screenshot shows the 'Main' tab of a configuration window. At the top, there are two tabs: 'Main' (active) and 'QoS'. Below the tabs, the 'Topic source' is set to 'Select from ROS network' with a dropdown arrow. The 'Topic' field contains '/joint_commands' and a 'Select' button is to its right. The 'Message type' field contains 'or_msgs/JointState'.

Topic = /joint_commands, Message type = sensor_msgs/JointState.

Main

QoS

Topic source: Specify your own

Topic: /joint_states

Message type: sensor_msgs/JointState

Select

Topic = /joint_states, Message type = sensor_msgs/JointState.

Screenshot 4: Bus Selector configurations

Block Parameters: Bus Selector

Select nested buses and signals from the input bus.

Elements in the bus

data

data_SL_Info

Output elements

data

☐ Output as virtual bus

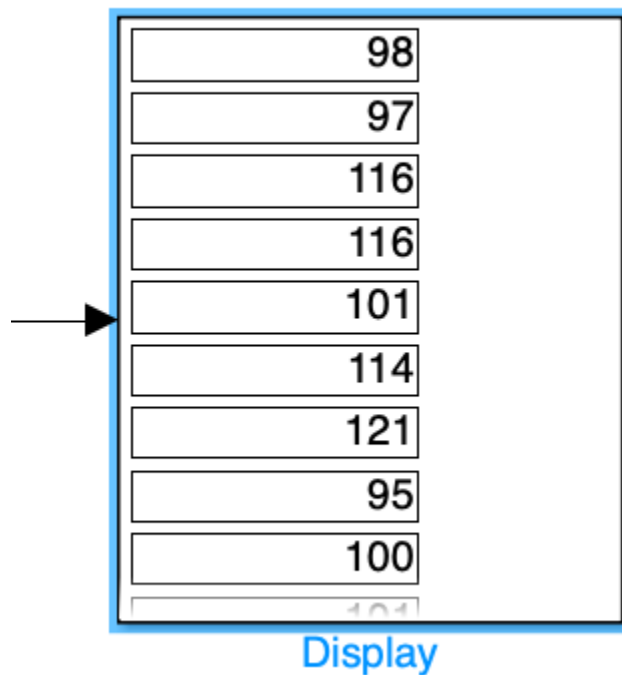
Help

Select the 'data' from the 'elements in the bus' section and click on this button



at the top. Then, from the 'output elements', delete anything else showing except 'data'.

Screenshot 5: Display block showing received data



Display shows the received `/detections` message as ASCII numbers (the string `"battery_detected"`). This proves the subscriber works.

Screenshot 6: Fake YOLO publisher script output

```
setenv('ROS_DOMAIN_ID', '50')

node = ros2node("/fake_yolo");
pub = ros2publisher(node, "/detections", "std_msgs/String");

msg = ros2message("std_msgs/String");
msg.data = 'battery_detected';

fprintf('Fake YOLO node started – sending "battery_detected" every 2 seconds\n');

while true
    send(pub, msg);
    fprintf('Sent: %s\n', msg.data);
    pause(2);
end
Fake YOLO node started – sending "battery_detected" every 2 seconds
Sent: battery_detected
Sent: battery_detected
Sent: battery_detected
Sent: battery_detected
Sent: battery_detected
Sent: battery_detected
Sent: battery_detected
Sent: battery_detected
Sent: battery_detected
```

Fake YOLO node publishing test messages to /detections.

4. How to run and verify

1. Start the fake YOLO node (in MATLAB command window):

```
setenv('ROS_DOMAIN_ID', '50');
```

```
node = ros2node("/fake_yolo");
```

```
pub = ros2publisher(node, "/detections", "std_msgs/String");
```

```

msg = ros2message("std_msgs/String");

msg.data = 'battery_detected';

fprintf('Fake YOLO node started – sending "battery_detected" every 2
seconds\n');

while true

    send(pub, msg);

    fprintf('Sent: %s\n', msg.data);

    pause(2);

end

```

2. Open and run the Simulink model `ur3_ros2_bridge_v2.slx`.
3. Observe that the Display block shows ASCII numbers (message received).

5. How this helps the project

- Provides a tested ROS2 bridge that can be directly integrated with the real YOLO node (once the CV team provides the actual detection message type).
 - Establishes the communication pattern for the whole control system: perception → control → robot → state feedback.
 - Allows us to later replace the dummy publishers with real joint commands from the UR3 model and real joint states from the robot simulation.
 - Serves as a reference for other team members who need to add ROS2 communication in Simulink.
-

6. Next steps after this documentation

- Replace the fake YOLO node with the CV team's actual YOLO node (using the correct detection message type).
 - Replace the dummy JointState sources with real joint angles from the UR3 Simulink model (using Bus Assignment blocks).
 - Implement a control algorithm (e.g., simple pick-and-place) that uses detection data to generate joint commands.
-